In [ ]:

```
                                                    Practical No: 01


Name : Dhanke Anuj Mahadev
Roll No : 20
Batch : A1
Problem Statement : Predict the price of the Uber ride from a given pickup point to the
EvaluatethemodelsandcomparetheirrespectivescoreslikeR2,RMSE,etc.
```

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [2]:

```
df = pd.read_csv("uber.csv")
```

In [3]:

```
df
```

Out[3]:

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085
...	...	...	...	...	...	...
199995	42598914	2012-10-28 10:49:00.00000053	3.0	2012-10-28 10:49:00 UTC	-73.987042	40.739367
199996	16382965	2014-03-14 01:09:00.0000008	7.5	2014-03-14 01:09:00 UTC	-73.984722	40.736837
199997	27804658	2009-06-29 00:42:00.00000078	30.9	2009-06-29 00:42:00 UTC	-73.986017	40.756487
199998	20259894	2015-05-20 14:56:25.0000004	14.5	2015-05-20 14:56:25 UTC	-73.997124	40.725452
199999	11951496	2010-05-15 04:08:00.00000076	14.1	2010-05-15 04:08:00 UTC	-73.984395	40.720077

200000 rows × 9 columns

In [4]:

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
df.head()
```

Out[4]:

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude	dro
0	24238194	2015-05-07 19:52:06.0000003	7.5	2015-05-07 19:52:06 UTC	-73.999817	40.738354	
1	27835199	2009-07-17 20:04:56.0000002	7.7	2009-07-17 20:04:56 UTC	-73.994355	40.728225	
2	44984355	2009-08-24 21:45:00.00000061	12.9	2009-08-24 21:45:00 UTC	-74.005043	40.740770	
3	25894730	2009-06-26 08:22:21.0000001	5.3	2009-06-26 08:22:21 UTC	-73.976124	40.790844	
4	17610152	2014-08-28 17:47:00.000000188	16.0	2014-08-28 17:47:00 UTC	-73.925023	40.744085	

In [5]:

```
df.tail()
```

Out[5]:

	Unnamed: 0	key	fare_amount	pickup_datetime	pickup_longitude	pickup_latitude
199995	42598914	2012-10-28 10:49:00.00000053	3.0	2012-10-28 10:49:00 UTC	-73.987042	40.739367
199996	16382965	2014-03-14 01:09:00.0000008	7.5	2014-03-14 01:09:00 UTC	-73.984722	40.736837
199997	27804658	2009-06-29 00:42:00.00000078	30.9	2009-06-29 00:42:00 UTC	-73.986017	40.756487
199998	20259894	2015-05-20 14:56:25.0000004	14.5	2015-05-20 14:56:25 UTC	-73.997124	40.725452
199999	11951496	2010-05-15 04:08:00.00000076	14.1	2010-05-15 04:08:00 UTC	-73.984395	40.720077

In [6]:

```
df.shape
```

Out[6]:

```
(200000, 9)
```

In [7]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200000 entries, 0 to 199999
Data columns (total 9 columns):
 #   Column             Non-Null Count   Dtype  
---  ------             --------------   -----  
 0   Unnamed: 0         200000 non-null  int64  
 1   key                200000 non-null  object 
 2   fare_amount        200000 non-null  float64
 3   pickup_datetime    200000 non-null  object 
 4   pickup_longitude   200000 non-null  float64
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
 5   pickup_latitude    200000 non-null  float64
 6   dropoff_longitude  199999 non-null  float64
 7   dropoff_latitude   199999 non-null  float64
 8   passenger_count    200000 non-null  int64
dtypes: float64(5), int64(2), object(2)
memory usage: 13.7+ MB
```

In [8]:

```
df.describe()
```

Out[8]:

	Unnamed: 0	fare_amount	pickup_longitude	pickup_latitude	dropoff_longitude	dropoff_latitu
count	2.000000e+05	200000.000000	200000.000000	200000.000000	199999.000000	199999.0000
mean	2.771250e+07	11.359955	-72.527638	39.935885	-72.525292	39.9238
std	1.601382e+07	9.901776	11.437787	7.720539	13.117408	6.7948
min	1.000000e+00	-52.000000	-1340.648410	-74.015515	-3356.666300	-881.9855
25%	1.382535e+07	6.000000	-73.992065	40.734796	-73.991407	40.7338
50%	2.774550e+07	8.500000	-73.981823	40.752592	-73.980093	40.7530
75%	4.155530e+07	12.500000	-73.967154	40.767158	-73.963658	40.7680
max	5.542357e+07	499.000000	57.418457	1644.421482	1153.572603	872.6976

In [9]:

```
df.isnull().sum()
```

Out[9]:

```
Unnamed: 0           0
key                  0
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    1
dropoff_latitude     1
passenger_count      0
dtype: int64
```

In [10]:

```
df["pickup_datetime"] = pd.to_datetime(df["pickup_datetime"])
```

In [11]:

```
df.dropna(inplace=True)
```

In [12]:

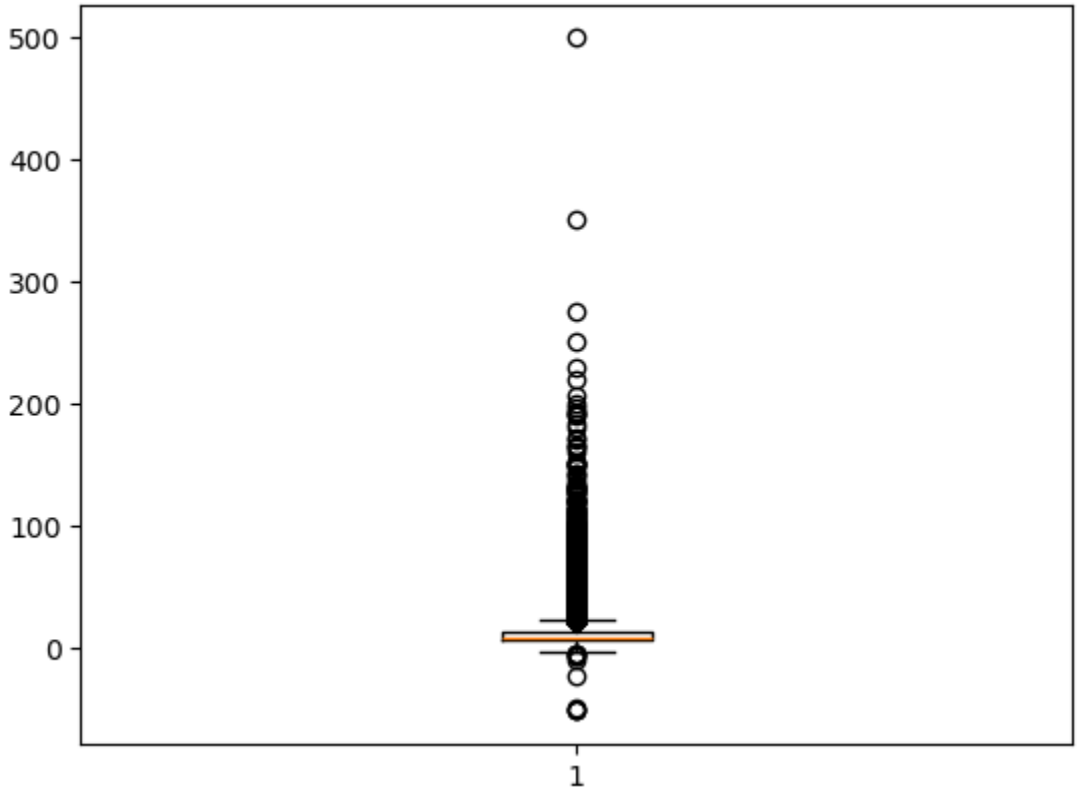
```
plt.boxplot(df['fare_amount'])
```

Out[12]:

```
{'whiskers': [<matplotlib.lines.Line2D at 0x1bd0559dd10>,
  <matplotlib.lines.Line2D at 0x1bd05524550>],
 'caps': [<matplotlib.lines.Line2D at 0x1bd05524690>,
  <matplotlib.lines.Line2D at 0x1bd055247d0>],
 'boxes': [<matplotlib.lines.Line2D at 0x1bd0559c2d0>],
 'medians': [<matplotlib.lines.Line2D at 0x1bd05524910>],
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

```
'fliers': [<matplotlib.lines.Line2D at 0x1bd05524a50>],
'means': []}
```



In [13]:

```
q_low = df["fare_amount"].quantile(0.01)
q_hi = df["fare_amount"].quantile(0.99)

df = df[(df["fare_amount"] < q_hi) & (df["fare_amount"] > q_low)]
```

In [14]:

```
df.isnull().sum()
```

Out[14]:

```
Unnamed: 0           0
key                  0
fare_amount          0
pickup_datetime      0
pickup_longitude     0
pickup_latitude      0
dropoff_longitude    0
dropoff_latitude     0
passenger_count      0
dtype: int64
```

In [15]:

```
from sklearn.model_selection import train_test_split
```

In [16]:

```
X = df.drop("fare_amount", axis=1)
y = df['fare_amount']
```

In [17]:

```
X['pickup_datetime'] = pd.to_numeric(pd.to_datetime(X['pickup_datetime']))
X = X.loc[:, X.columns.str.contains('Unnamed')]
```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF

In [18]:

```
x_train, x_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)
```

In [19]:

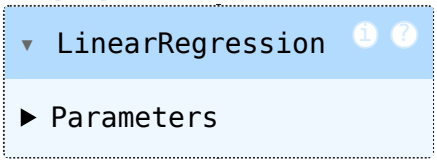
```
from sklearn.linear_model import LinearRegression
```

In [20]:

```
lrmodel = LinearRegression()

lrmodel.fit(x_train, y_train)
```

Out[20]:



In [21]:

```
predict = lrmodel.predict(x_test)
```

In [22]:

```
from sklearn.metrics import mean_squared_error

lrmodel.rmse = np.sqrt(mean_squared_error(predict, y_test))

print("RMSE error for the model is", lrmodel.rmse)
```

```
RMSE error for the model is 8.063863046328837
```

In [23]:

```
from sklearn.ensemble import RandomForestRegressor

rfrmodel = RandomForestRegressor(n_estimators=100, random_state=101)
```

In [ ]:

```
rfrmodel.fit(x_train, y_train)

rfrmodel_pred = rfrmodel.predict(x_test)
```

In [ ]:

```
rfrmodel_rmse = np.sqrt(mean_squared_error(rfrmodel_pred, y_test))

print("RMSE value for RandomForestis:", rfrmodel_rmse)
```

In [ ]:

```

```

Explore our developer-friendly HTML to PDF API Printed using PDFCrowd HTML to PDF